

Python Internals: Stages of Execution (The Python Execution Model)



Syntax error: syntax error occurs if the code is written in a way that it doesn't comply with the rules or laws of the python language.

Runtime error: runtime error occurs when the code is syntactically correct, but there are other issues with the code such as:

1. misspelled or incorrectly capitalized variable and function names.
 2. attempts to perform operations such as math operations on data of the wrong type.
 3. dividing by zero i.e. $1 / 0$
 4. attempts to use type conversion function such as `int` on a value that can't be converted to an `int` type.
- Runtime Error: An issue that occurs while the program is actively running, stopping the execution because it hits an impossible operation, such as “dividing by zero, which makes no sense in mathematics.”



Myths about the python:

1. python doesn't include the compilation phase
2. all errors are caught at runtime and they are caught line by line

A compiler translates the entire source code into machine code all at once before running it, while an interpreter translates and executes the code one line at a time as the program runs

1. as python interpreter executes the code line by line, so ideally the error should occur at this line, but it doesn't come here

4. but again this runtime error does not get detected by the python interpreter

```
"""
1. Example for Demonstrating stages of compilation.
2. Let's learn by debugging
"""
```

```
# Step 1
```

```
1 / 0
print() = None
if False
    n = "hello"
```

this gets detected during the scanning phase

2. this syntax error is showing at this line

```
Cell In[2], line 9
```

```
n = "hello
    ^
```

SyntaxError: unterminated string literal (detected at line 9)

```
# Step 2
```

```
1 / 0
print() = None
if False
    n = "hello"
```

5. this syntax error gets detected first

3. syntax error is corrected here

```
Cell In[3], line 3
```

```
print() = None
    ^
```

SyntaxError: cannot assign to function call here. Maybe you meant '==' instead of '='?

```
# Step 3
```

```
1 / 0
print()
```

6. now, this error is also get resolved

7. but again the error 1 / 0 doesn't get detected by the python interpreter instead, the interpreter catches this another error.

```
if False
    n = "hello"
```

this error gets detected in the parsing phase

```
Cell In[4], line 4
    if False
        ^
SyntaxError: expected ':'
```

```
# Step 4
1 / 0
print()
if False:
    n = "hello"
```

8. now, we have again corrected this syntax error

but, now this runtime error is caught by the interpreter

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[5], line 2
      1 # Step 4
----> 2 1 / 0
      3 print()
      4 if False:

ZeroDivisionError: division by zero
```

```
# Step 5 => Success
1 / 0.1
print()
if False:
    n = "hello"
```

What happens when you install Python ?

- When you install Python on your system (e.g., using the official installer from python.org), a **number of components** are set up to **create a complete Python development environment**. Let's look into some important python modules which gets installed .

✓ 1. **Python Interpreter (CPython)** - The core component that parses, compiles, and executes your Python code. - It converts .py files into bytecode and runs them via the **Python Virtual Machine (PVM)**.

✓ 2. **Standard Library** - A huge collection of modules (in-built packages) that provide: File I/O, Regular expressions (re), Networking (socket, http), Math (math, statistics) etc

✓ 3. **Bytecode Compiler** - .py files are automatically compiled to .pyc files and stored in the **pycache** directory. - This helps speed up execution.

Is Python Compiled or Interpreted?

- Python is interpreted. Let's understand in details.
- The Python interpreter software you download from python.org is called CPython because it's written in C.

runs the
bytecode
using PVM

python interpreter CPython does the work of compiler
in python

- When you run a Python program, the interpreter first compiles it to bytecode and then runs the bytecode. So you could say that Python is compiled.
- The CPython interpreter really is an interpreter. But it also is a compiler.
- Python must go through a few stages before ever running the first line of code: 1.

also called as Lexical Analysis Scanning and 2. Parsing also called as syntax analysis

NOTE

1. Interpreter - General term for any system that executes code without compiling to native machine code first
2. CPython - The full Python interpreter, written in C
3. PVM - The runtime engine inside CPython that runs bytecode

-
- Sample Python code for Analysis :

```
x = 5 + 3
```

The above code is saved in .py file in local system.

6.1. Lexical Analysis and Tokenization - Scanning Phase

during the scanning
phase, Syntax errors
and Indentation errors
gets detected

- The first step in the execution process is **lexical analysis**, also known as **tokenization**.
- During this phase, the source code is converted into a sequence of **tokens**.
- **Tokens:**

```
[ x (identifier) , = (operator), 5 (literal), + (operator), 3  
(literal) ]
```

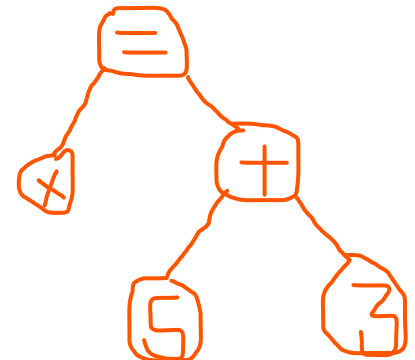
assignment operator

6.2. Syntax Analysis and Abstract Syntax Tree (AST) - Parsing Phase

- During this phase, the sequence of tokens is analyzed to determine its grammatical structure.
- The output of this phase is an Abstract Syntax Tree (AST), which represents the hierarchical structure of the source code.

Abstract
Syntax Tree

```
Assignment  
├─ Target: x  
└─ Value  
    ├─ Left: 5  
    └─ Right: 3  
        └─ Operator: +
```



6.3. Bytecode Compilation

The Python Virtual Machine (PVM) is a part of CPython

- The AST is then compiled into bytecode, which is a low-level, platform-independent representation of the source code.
- Bytecode is a set of instructions that can be efficiently executed by the Python Virtual Machine (PVM).
- Generates a .pyc file
- Bytecode Examples

```
1 LOAD_CONST 5
2 LOAD_CONST 3
3 BINARY_ADD
4 STORE_NAME x
```

load the
constant
value 5

Bytecode for x = 5 + 3 , looks like this

binary addition

```
101 <----- 5
+ 011 <-----3
-----
1000 <-----8
```

```
code_obj = compile('x = a+ b', '', 'exec')
print(type(code_obj))
# <class 'code'>

import dis
dis.dis(code_obj)
```

```
<class 'code'>
0          RESUME          0

1          LOAD_NAME       0 (a)
          LOAD_NAME       1 (b)
          BINARY_OP        0 (+)
          STORE_NAME       2 (x)
          RETURN_CONST     0 (None)
```

run this
code to see
it's
correspondin
g bytecode

6.4. Execution by the Python Virtual Machine (PVM)

- PVM is the Python Interpreter that converts the Python byte code into machine-executable code.
- PVM interpreter reads and executes the given file line by line.
- CPU runs the native machine instructions triggered by the PVM
- It manages memory, automatically cleaning up unused objects using a garbage collector
- PVM also provides thread safety With a Global Interpreter Lock (GIL) that comes with Cpython.

because the cpython
implementation is not
thread safe.

this a mutex lock
and this makes
sure that our
bytecode
gets executed by
a single thread at
one time. That is,
no multithreading
allowed.

as our cpython is
not thread safe, so
it's the
responsibility of
PVM to take care
of that.

As, the code was:-

```

L1 → 1/0
L2 → print() = None
L3 → if False
L4 → n = "hello"

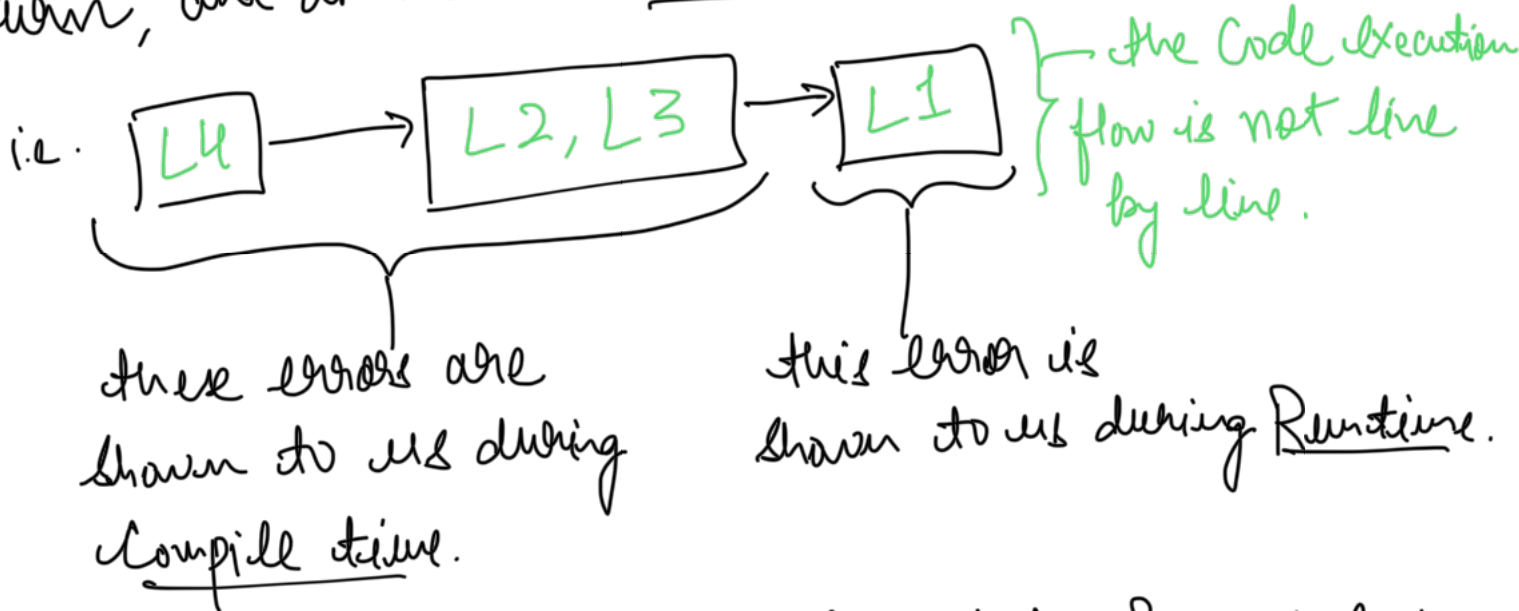
```

this error gets detected during Runtime.

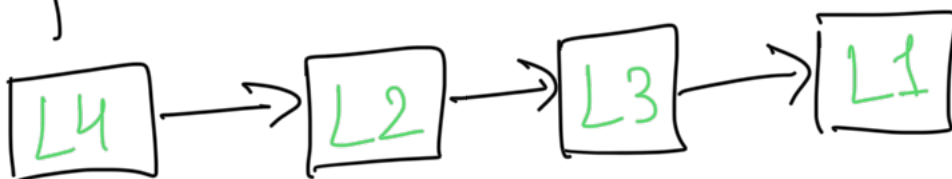
these errors gets detected during the Parsing Phase.

this error gets detected during the Scanning Phase.

Now, in the stages of execution of the Python Code, we have seen that the first comes the Scanning phase i.e., it means errors that belongs to the scanning phase would be detected first, then comes the Parsing phase i.e., errors that belongs to parsing phase would get detected in the second turn, and at the end Runtime errors would get detected.



So, the final order to execution of the Python code is:-



So we have proved that the above Myth of Python execution is True.